Documentation of UR5 robot
U2IS - ENSTA Paris
PIE - 2025/2026

Control & Robotic Arm Group

Diego Magno MONTANS ZUCHERAM	diego-magno.montans@ensta.fr
Gustavo KURATOMI PEREIRA E COSTA	gustavo.kuratomi@ensta.fr
Thiago ROSA CARVALHO	thiago.rosa@ensta.fr
Yasmine DHOUIB	yasmine.dhouib@ensta.fr

Contents

1 Introduction **4**
1.1 Project Repository . . . 5

2 System Requirements **5**

3 System Configuration **5**

4 Programation and Connection **6**

5 UR5 Setup **13**
5.1 Setup Function Overview . . . 13
5.2 Robot Model Loading . . . 14
5.3 ROS2 Remote Device Configuration . . . 14
5.4 Tool Center Point (TCP) Definition . . . 14
5.5 UR ROS2 Node Creation . . . 15
5.6 Inverse Kinematics Initialization . . . 15
5.7 End-Effector Frame and IK Weights . . . 15
5.8 Summary . . . 15

6 Frame Calibration using AX = XB **16**
6.1 Problem Formulation . . . 16
6.2 Data Acquisition . . . 16
6.3 Solution Method . . . 17
6.4 Resulting Transformation . . . 18
6.5 Role in the System . . . 18
6.6 Coordinate Conversion for Testing . . . 18

7 MATLAB Control Implementation **19**
7.1 ROS2 Communication and Data Acquisition . . . 19
7.2 Throttling and Manual Triggering . . . 20
7.3 Inverse Kinematics Formulation . . . 20
7.4 Improved IK Solution Selection . . . 21
7.5 Approach Strategy for Safer Picking . . . 21
7.6 Command Execution . . . 22
7.7 Tool Output and Gripper Actuation . . . 22
7.8 Integration with Camera-Based System . . . 23
7.9 Overall Control Logic . . . 23

8 ROS2 Integration **23**
8.1 Network Configuration . . . 24
8.2 ROS2 Initialization in MATLAB . . . 24
8.3 Subscriber Implementation . . . 24
8.4 Callback-Based Data Handling . . . 25
8.5 Service Client for Tool Control . . . 25

8.6 Communication Verification 26
8.7 Role in the Control Architecture 26

9 Conclusion **26**

1. Introduction

This document describes the development and configuration of the **PIE Pick and Place** project, implemented using the **UR5 robotic manipulator** as the main hardware platform.

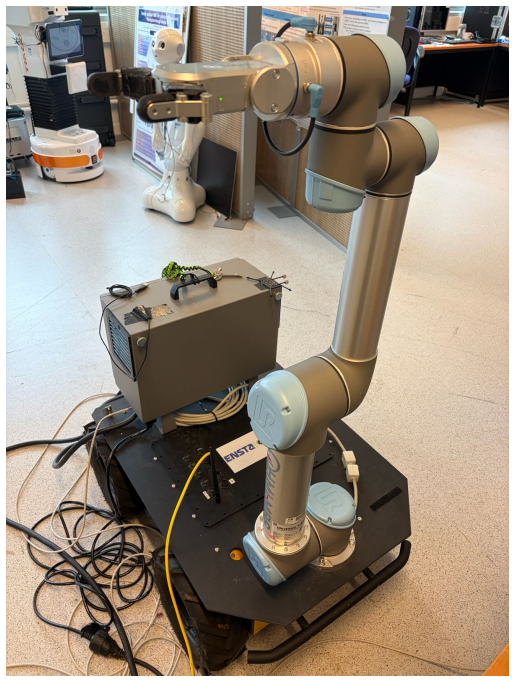

Figure 1: UR5

The objective of this project is to implement a complete pick-and-place application integrating computational control, robot communication, and motion execution. The system combines robot configuration, control strategies implemented in MATLAB, and integration with ROS2 to enable structured task execution.

This documentation presents:

- The configuration procedure required to connect and operate the UR5 robotic arm from a computer;
- The implementation of robot control using MATLAB;
- The integration architecture between MATLAB and ROS2;
- The system requirements, including software dependencies and configuration constraints.

The purpose of this document is to provide a structured technical reference that enables reproduction of the project setup, understanding of the system architecture, and future extensions of the implemented solution.

The document is organized as follows:

- **System Requirements:** Hardware specifications and software dependencies required to run the system, including operating system, ROS2 distribution, and MATLAB toolboxes.

- **System Configuration:** Installation steps, network setup, and procedures to establish communication between the computer and the UR5 robot.
- **UR5 Configuration:** Network setup, operational mode selection, and Tool Center Point (TCP) definition for accurate task execution, as well as robot initialization procedures.
- **Programming and Connection:** Step-by-step procedure to start the robot, load the correct installation and program, configure external control via MATLAB URCap, and launch the ROS2 driver.
- **UR5 Setup:** MATLAB-side initialization, including robot model loading, ROS2 device configuration, node creation, and inverse kinematics setup.
- **Frame Calibration using AX = XB:** Formulation and solution of the hand-eye calibration problem used to estimate the transformation between the camera and the robot base frame.
- **MATLAB Control Implementation:** Description of the control pipeline, including coordinate transformations, pose construction, inverse kinematics, and command execution.
- **ROS2 Integration:** Node architecture, network configuration, subscriber implementation, and data exchange between MATLAB and external processes.

1.1. Project Repository

The source code developed for this project, including MATLAB control scripts and ROS2 integration tools for the UR5 robot, is available in the following GitHub repository:
https://github.com/gustavokpc/ROS2_UR5_PIE

2. System Requirements

- Ubuntu 22.04 LTS (Ubuntu Pro)
- ROS2 Humble
- MATLAB (with Robotics System Toolbox™,Robotics System Toolbox™ Support Package for Universal Robots UR Series Manipulators, ROS Toolbox)
- UR ROS2 Driver

3. System Configuration

To start the configuration, connect the ETHERNET cable from the robot to the computer.
For this robot, we are using ROS2 Humble. Firstly, install the following repository to make the connection:
Universal Robots ROS2 Driver Github

To install the MATLAB dependencies, refer to the the following website for the UR5 robot:
Getting Started with Connecting and Controlling a UR5e Cobot from Universal Robots using ROS 2

The following link website can also help with the hardware setup:

Hardware Setup for UR Series Cobots for ROS 2

After accessing the link above, follow the tutorial provided on the website to install the required program for controlling the robot using MATLAB. During this step, a USB flash drive will be required to transfer the necessary files to the robot controller.

4. Programation and Connection

Before starting the robot initialization procedure, it is necessary to correctly configure the network settings of the computer that will be used to control the UR5.

Network Configuration

The computer must be configured with a static IP address compatible with the robot network. In our setup, the IPv4 configuration was manually defined as follows:

- IP Address: 147.250.35.73
- Netmask: 255.255.255.0
- Gateway: 147.250.35.245

This configuration ensures that the computer and the robot belong to the same subnet, allowing direct communication between them.

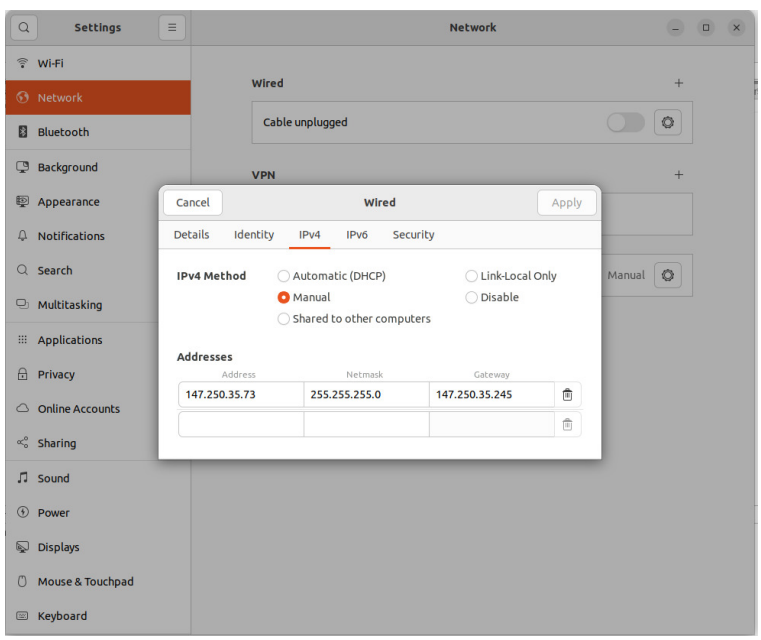

Figure 2: Manual IPv4 configuration of the control computer.

Additionally, in order to enable communication not only with the robot but also with other project components (such as the cage system and the camera module via ROS2), a network switch was used.

The switch allows multiple devices to be connected simultaneously within the same network, ensuring reliable communication between:

- the control computer (MATLAB),
- the robot controller,
- the perception system (camera),
- and the external instruction system (cage).

This setup was essential to guarantee proper ROS2 communication and data exchange between all subsystems.

Firstly, turn on the robot using the tablet by pressing the green power button.

In the initial screen, click on **"Setup Robot"**.

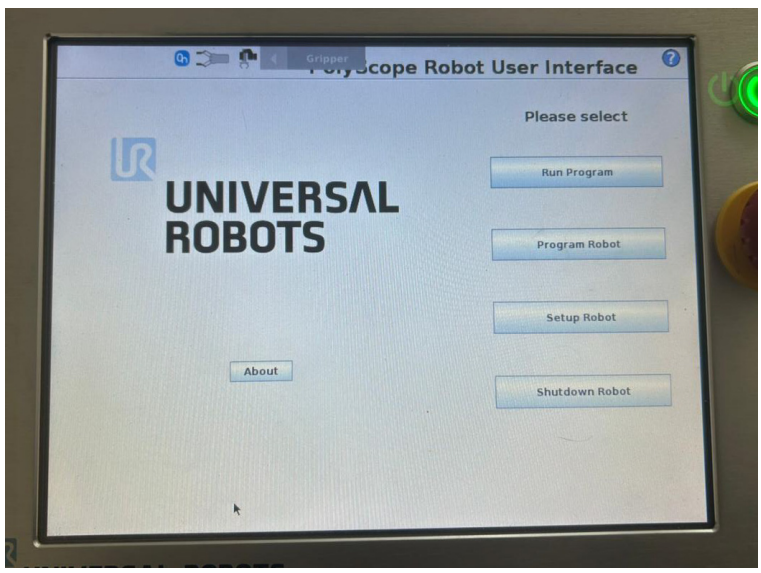

Figure 3: Initial Screen

In the Setup Robot screen, click on **"Initialize Robot"**.

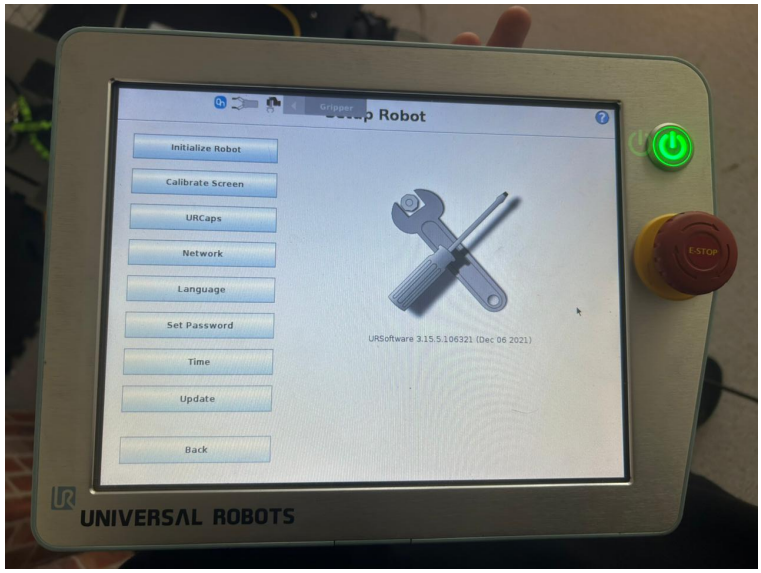

Figure 4: Setup Robot

After turning on the robot, press the **"Turn On"** button and then press **"START"** as shown in figures 5 and 6.

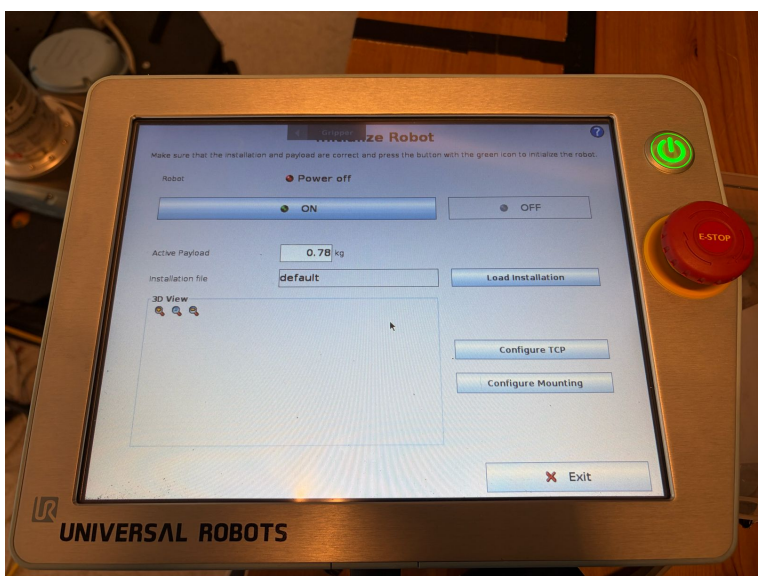

Figure 5: Turn On button on the robot tablet

Figure 6: Start button on the robot tablet

This action releases the robot brakes. Before pressing start, make sure the robot is in a stable and safe position.

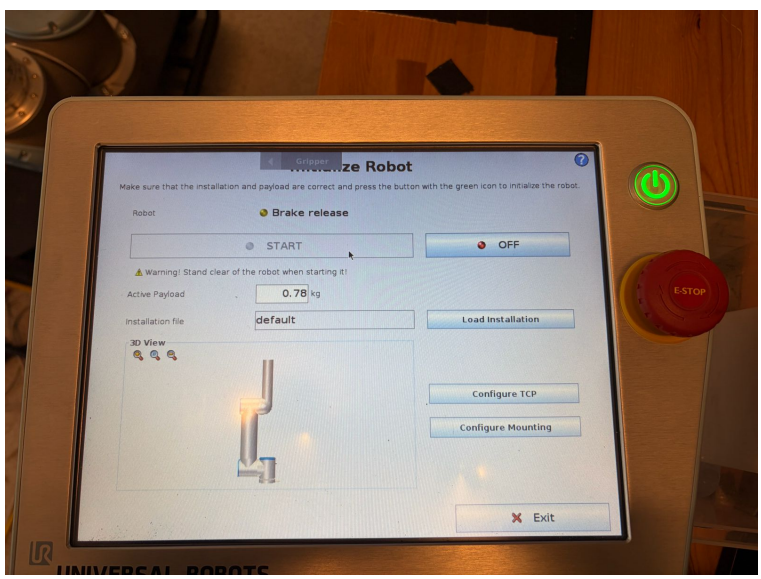

Figure 7: Brake release procedure.

Once the robot is initialized, the system should display the **"Normal"** status, indicating that the robot is ready for operation (Figure 8)

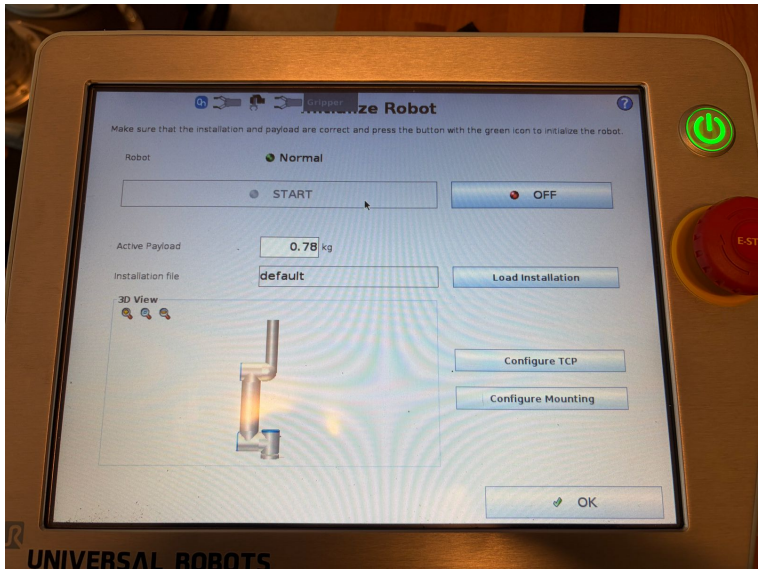

Figure 8: Robot in normal operating state

Next, load the installation that was previously transferred to the robot controller.

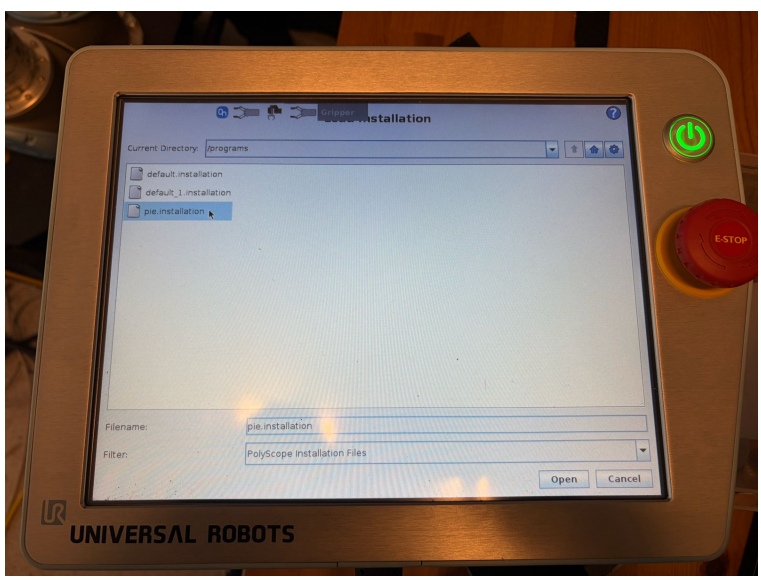

Figure 9: Loading the installation file

After loading the installation, click *"EXIT"*. It will come back to the image 3. Click *"Program Robot"* and *"Load Program"* afterwards. Then, proceed to load the program installed in Section 3.

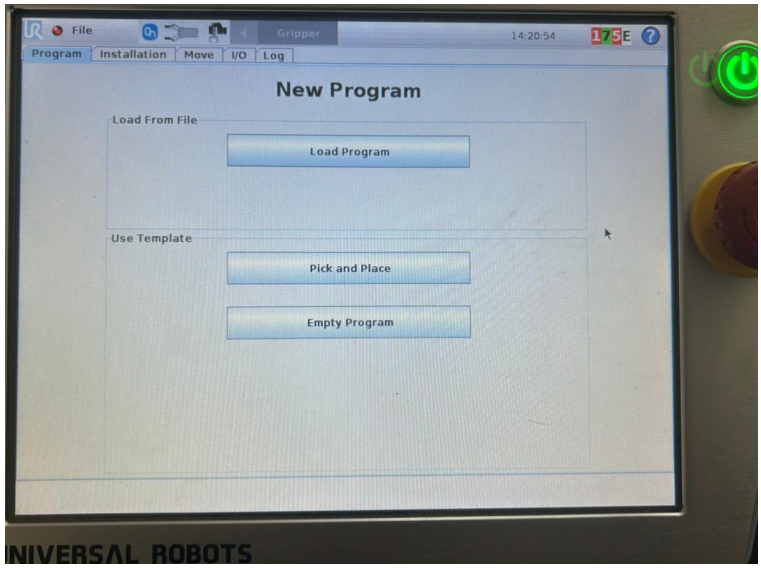

Figure 10: Loading the robot program – New Program screen

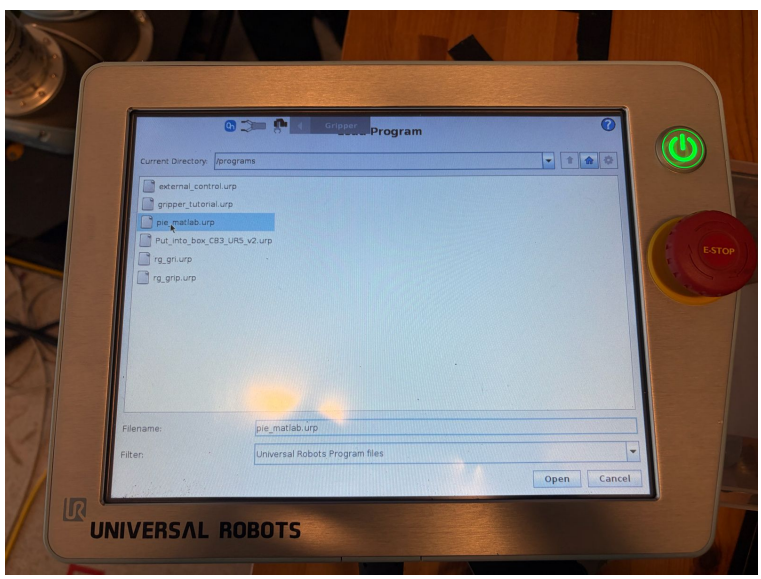

Figure 11: Loading the robot program – Load Program screen

Note: Initially, the `pie_matlab.urp` program was used. However, due to stability and communication issues, it was replaced by the `external_control.urp` program, which provided more reliable integration with MATLAB and ROS2.

If everything has been configured correctly, the robot status should appear as shown below.

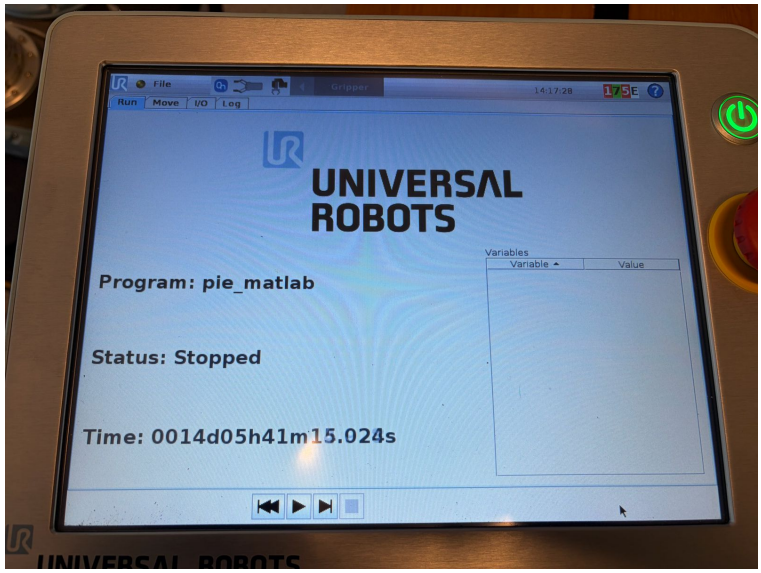

Figure 12: Robot program loaded and ready to start

After this step, it is necessary to configure the IP address of the computer that will control the robot. To do this, open the **"Installation"** tab and select **"External Control"** and then the same for **"MATLAB URCap for External Control"**.

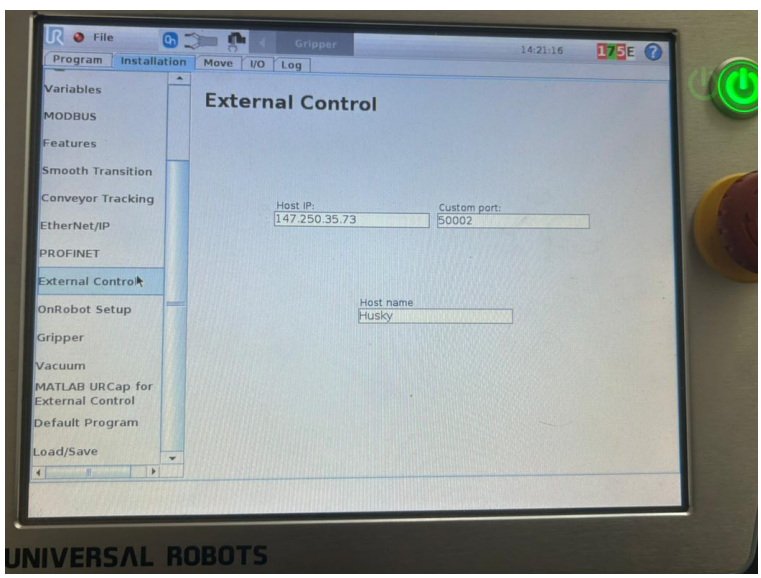

Figure 13: External control configuration

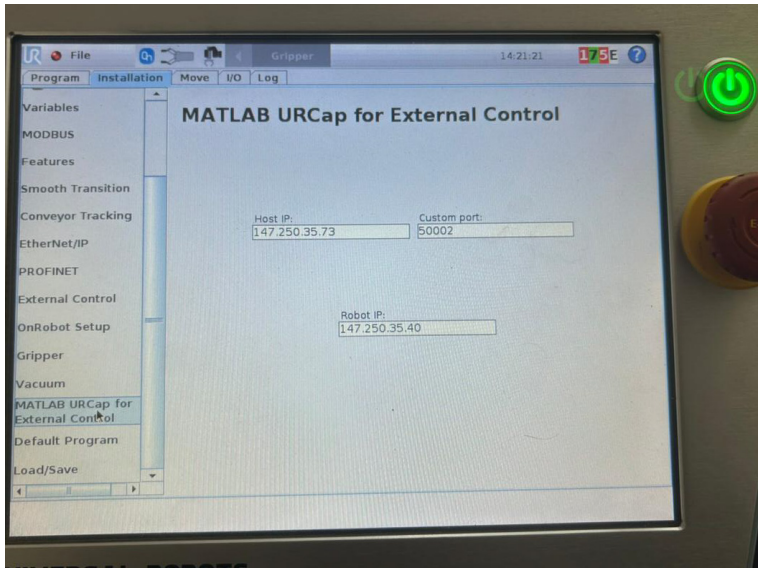

Figure 14: MATLAB URCap external control configuration

In this section, enter the IP address of the computer that will be used to control the robot from MATLAB.

After configuring the IP address and connecting the computer to the robot using the Ethernet cable, the ROS2 driver must be launched from the computer.

First, source the ROS2 environment and start the Universal Robots driver using the robot IP address:

```
source /opt/ros/humble/setup.bash

ros2 launch ur_robot_driver ur_control.launch.py
ur_type:=ur5 robot_ip:='147.250.35.40' launch_rviz:=false
```

Once the driver is running, MATLAB can be opened on the same computer to establish the external control connection with the robot.

5. UR5 Setup

This section describes the initialization of the UR5 manipulator on the MATLAB side. All heavy setup steps (robot model loading, ROS2 device configuration, node creation, tool definition, and inverse kinematics initialization) are grouped into a single function `setupUR5()`, which returns a context structure `ctx` reused across the project.

5.1. Setup Function Overview

The setup function returns a MATLAB structure `ctx` containing all persistent objects required to interact with the UR5:

- the UR5 kinematic model as a `RigidBodyTree`,

- a custom Tool Center Point (TCP) frame attached to the robot,
- the ROS2 interface node to the robot controller,
- an inverse kinematics (IK) solver created once,
- fixed parameters such as end-effector name and IK weights.

5.2. Robot Model Loading

The first step is to load the UR5 kinematic model:

```
ctx.ur5 = loadrobot('universalUR5');
```

This model is used both for forward/inverse kinematics computations and for defining the robot structure passed to the ROS2 control interface.

5.3. ROS2 Remote Device Configuration

The UR5 controller (or the machine directly connected to it) is accessed remotely via SSH using MATLAB's `ros2device`. The following parameters define the connection:

```
username = 'student';
password = 'student';

ROS2Folder = '/opt/ros/humble';
ROS2Workspace = '/opt/ros/humble';

ROS2DeviceAddress = '147.250.35.73';
```

A ROS2 device object is then created and configured:

```
device = ros2device(ROS2DeviceAddress, username, password);
device.ROS2Folder = ROS2Folder;
device.ROS2Workspace = ROS2Workspace;
```

This step ensures that MATLAB knows where ROS2 is installed on the remote machine and can properly interact with ROS2 nodes running in that environment.

5.4. Tool Center Point (TCP) Definition

In the physical setup, a gripper is mounted on the robot flange. Therefore, the default `tool0` frame does not correspond to the actual contact point of the robot.

To account for this, a custom TCP frame is explicitly added to the robot model as a fixed rigid body:

```
tcp = rigidBody('tcp');
jnt = rigidBodyJoint('tcp_fixed','fixed');

offset = trvec2tform([0 0 0.21]);
setFixedTransform(jnt, offset);
```

```
tcp.Joint = jnt;
addBody(ctx.ur5, tcp, 'tool0');
```

This transformation defines the offset between the flange (`tool0`) and the tip of the gripper. As a result, all kinematic computations are performed directly at the gripper tip, improving accuracy in task execution.

5.5. UR ROS2 Node Creation

Once the model and TCP are defined, a dedicated ROS2 interface node is created using `urROS2Node`:

```
ctx.ur = urROS2Node('RigidBodyTree', ctx.ur5);
```

This object is responsible for communicating with the UR driver through ROS2. It provides the bridge between MATLAB computations (e.g., IK solutions, trajectories) and command transmission to the UR5 controller.

5.6. Inverse Kinematics Initialization

Inverse kinematics is required to convert Cartesian target poses into joint configurations. Since IK initialization is computationally expensive, the solver is created only once:

```
ctx.ik = inverseKinematics("RigidBodyTree", ctx.ur5);
```

This design avoids re-creating the solver at each command iteration and ensures faster runtime performance.

5.7. End-Effector Frame and IK Weights

With the TCP explicitly defined in the model, the end-effector reference used for kinematics becomes:

```
ctx.eeName = "tcp";
```

This ensures that all Cartesian commands are defined with respect to the gripper tip, rather than the robot flange.

Finally, the IK solver weights are defined as:

```
ctx.weights = [1 1 1 1 1 1];
```

which assigns equal importance to position and orientation components during optimization.

5.8. Summary

The function `setupUR5()` centralizes all UR5 initialization steps, including robot model loading, TCP definition, ROS2 configuration, and IK setup. It returns a single context structure `ctx`, reused by all motion and communication functions in the project.

This approach ensures consistency across the control pipeline, improves computational efficiency, and guarantees that all commands are expressed relative to the actual end-effector of the robot.

6. Frame Calibration using AX = XB

In order to accurately execute pick-and-place operations based on visual data, it is necessary to determine the spatial relationship between the camera reference frame and the robot base frame. This transformation is not directly known and must be estimated through a calibration procedure.

6.1. Problem Formulation

Let us define the following reference frames:

- b: robot base frame
- c: camera frame
- g: observed object (or marker) frame

The goal of the calibration is to estimate the rigid transformation $X_{b,c} \in SE(3)$, which maps coordinates from the camera frame to the robot base frame.

To achieve this, we use multiple observations of the robot and the object, leading to the classical hand-eye calibration problem expressed as:

$$A_i X = X B_i \tag{I}$$

where:

- A_i represents the relative motion of the robot between two poses, expressed in the robot base frame,
- B_i represents the corresponding relative motion observed by the camera,
- X is the unknown transformation between the camera frame and the robot base frame.

6.2. Data Acquisition

The calibration process relies on collecting a set of paired observations of the robot and the camera.

In this project, approximately 20 distinct robot configurations were recorded. For each configuration, two measurements were acquired:

- The robot pose $T_{b,g}$, obtained from the UR5 controller, expressed in the robot base frame.
- The corresponding pose $T_{c,g}$, obtained from the vision system, expressed in the camera frame.

Each pose corresponds to a different spatial configuration of the robot, ensuring sufficient motion diversity for the calibration process. This diversity is essential for obtaining a well-conditioned estimation of the transformation.

From these measurements, homogeneous transformation matrices are constructed for both the robot and the camera. These matrices are then used to form motion pairs for the calibration procedure.

6.3. Solution Method

The calibration problem is solved using a numerical optimization approach implemented in the script `AXXB.m`.

Instead of directly computing relative motions (A_i, B_i), the implementation estimates two transformations simultaneously:

- $X_{g,c}$: transformation between the gripper and the camera,
- $X_{b,c}$: transformation between the robot base and the camera.

The optimization enforces the consistency constraint:

$$T_{b,g}^{(i)} \cdot X_{g,c} = X_{b,c} \cdot T_{c,g}^{(i)} \tag{II}$$

for all recorded poses i.

The unknown transformations are parameterized using a 6D representation (Rodrigues vector for rotation and translation vector), resulting in a 12-dimensional optimization problem. The cost function minimizes the discrepancy between both sides of the equation across all samples:

- rotational error, measured using the Frobenius norm of rotation differences,
- translational error, weighted to balance its influence in the optimization.

The optimization is performed using `fminsearch`, which iteratively minimizes the global cost over all observations.

Outlier Rejection To improve robustness, an outlier rejection step is applied after an initial estimation. A score combining position and rotation errors is computed for each sample, and outliers are detected using a Median Absolute Deviation (MAD) criterion.

The calibration is then recomputed using only the inlier set, leading to a refined and more reliable estimate of the transformations.

Validation After optimization, the estimated transformations are validated by reconstructing the robot poses:

$$T_{b,g}^{\text{est}} = X_{b,c} \cdot T_{c,g} \cdot X_{g,c}^{-1} \tag{III}$$

The difference between the estimated and measured robot poses is evaluated in terms of:

- position error (in meters),
- rotation error (in degrees).

This validation step provides quantitative metrics to assess the quality of the calibration.

6.4. Resulting Transformation

The calibration yields a fixed homogeneous transformation:

$$X_{b,c} = \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix} \tag{IV}$$

where $R \in SO(3)$ is the rotation matrix and $t \in \mathbb{R}^3$ is the translation vector between the camera and the robot base.

This transformation is assumed to remain constant during operation and is reused throughout the control pipeline.

6.5. Role in the System

The estimated transformation $X_{b,c}$ plays a critical role in the system, as it allows converting positions detected by the camera into the robot reference frame.

Given a pose $T_{c,g}$ expressed in the camera frame, the corresponding pose in the robot base frame is computed as:

$$T_{b,g} = X_{b,c} \cdot T_{c,g} \tag{V}$$

This transformed pose is then used as input for the inverse kinematics solver described in the following section.

Without this calibration step, it would not be possible to correctly map visual detections to executable robot motions.

6.6. Coordinate Conversion for Testing

Once the calibration matrices were estimated, they were used in a MATLAB utility function named `coordConvert`. This function was responsible for converting a pose provided by the vision system in the cage reference frame into the pose format required by the robot controller.

The input pose was received in the form:

```
[rot_x rot_y rot_z rot_w X Y Z]
```

where the orientation is expressed as a quaternion and the position is expressed in the cage reference frame. Using the estimated calibration matrices, the function first constructs the homogeneous transformation of the detected object in the cage frame, and then maps it into the robot base frame according to:

$$T_{b,g} = X_{b,c}\, T_{c,g}\, X_{g,c}^{-1} \tag{VI}$$

Finally, the resulting homogeneous transformation is converted into the pose representation expected by the robot:

```
[tz ty tx x y z]
```

assuming the rotation convention:

$$R = R_z(t_z) R_y(t_y) R_x(t_x) \tag{VII}$$

This conversion was particularly useful during the testing phase, since the coordinates published by the cage system were still expressed in its own reference frame. Therefore, `coordConvert` provided an intermediate step to transform those poses into the robot reference frame before executing motion commands.

In the final version of the system, however, this conversion was no longer required during runtime. The cage system was updated to directly publish object coordinates already expressed in the robot base frame. As a result, `coordConvert` was kept only as a validation and testing tool, and was no longer part of the final online control pipeline.

7. MATLAB Control Implementation

This section describes the final MATLAB-based control pipeline used to command the UR5 robot from target positions received through ROS2.

During the development of the project, the MATLAB control architecture underwent several refinements. Initial versions of the implementation were able to receive target coordinates and command the robot, but some inverse kinematics solutions led to undesirable configurations, including self-collisions, abrupt joint changes, and unnatural motions. To improve robustness, the final implementation introduced additional strategies for solution selection and motion execution.

In the final system architecture, the target positions are directly provided in the robot base frame. Therefore, no coordinate transformation is required during runtime. The controller receives a 3D position vector $p \in \mathbb{R}^3$ (given as $[x\ y\ z]$), computes a feasible joint configuration through inverse kinematics, and sends the command to the robot.

7.1. ROS2 Communication and Data Acquisition

A ROS2 node is created in MATLAB to receive target positions:

```
rosNode = ros2node("/matlab_ur5_node");
```

A subscriber listens to the topic published by the external system:

```
sub = ros2subscriber(rosNode, "/instruction_cage", ...
    "geometry_msgs/Transform", ...
    @(msg) storePointThrottled(ctx, msg));
```

Only the translational components of the received message are used:

```
x = msg.translation.x;
y = msg.translation.y;
z = msg.translation.z;
```

These coordinates are already expressed in the robot base frame. In earlier testing stages, additional coordinate conversion steps (based on the AX = XB calibration described in Section 6) were required. However, in the final version of the system, this conversion is performed upstream, and MATLAB directly receives the target position in the correct reference frame.

7.2. Throttling and Manual Triggering

To avoid excessive command updates and to ensure stable operation, a throttling mechanism was introduced. Incoming target points are only accepted if a minimum time interval has elapsed since the last accepted point:

```
shared.throttle = 2.0; % seconds
```

This prevents the system from continuously reacting to high-frequency incoming data, which could cause unstable or unnecessary motion updates.

In addition, motion execution is not performed automatically as soon as a point is received. Instead, the latest accepted point is stored, and the user manually triggers execution by pressing Enter:

```
input("Enter = enviar para o último ponto | Ctrl+C = sair ", "s");
```

This design was particularly useful during testing, as it allowed controlled validation of each commanded motion before execution.

A locking mechanism was also added to prevent overlapping commands while the robot is still executing a previous trajectory.

7.3. Inverse Kinematics Formulation

The inverse kinematics computation is implemented in the function `computeIK_UR5`. The input is a Cartesian point:

```
p = [x y z];
```

From this point, the end-effector yaw is computed as:

```
yaw = atan2(p(2), p(1)) - pi/2;
```

The remaining orientation components are fixed, yielding the pose definition:

```
thetaZ = yaw;
thetaY = 0;
thetaX = -1.5708;
```

The corresponding homogeneous transformation is then constructed from the Z-Y-X rotation convention:

```
Rz = axang2rotm([0 0 1 thetaZ]);
Ry = axang2rotm([0 1 0 thetaY]);
Rx = axang2rotm([1 0 0 thetaX]);
R  = Rz * Ry * Rx;

T = eye(4);
T(1:3,1:3) = R;
T(1:3,4)   = [x; y; z];
```

This transformation defines the desired TCP pose in the robot base frame.

7.4. Improved IK Solution Selection

A major correction introduced during development concerned the selection of the inverse kinematics solution.

In early versions, a single IK solution was used directly. However, because the UR5 admits multiple valid joint configurations for the same Cartesian pose, some solutions caused problematic robot behavior, such as self-collisions, excessive joint motion, or unnatural trajectories.

To address this issue, the final implementation uses the robot's current joint configuration as the seed for IK:

```
try
    q0 = getJointConfiguration(ctx.ur);
catch
    q0 = homeConfiguration(ctx.ur5);
end
```

An initial solution is first computed:

```
[qSol, info] = ctx.ik(ctx.eeName, T, ctx.weights, q0);
```

Then, multiple additional candidate solutions are generated by perturbing the initial seed:

```
for k = 1:20
    qTest = ctx.ik(ctx.eeName, T, ctx.weights, q0 + 0.2*randn(size(q0)));
    ...
end
```

Among all candidates, the solution closest to the current robot configuration is selected according to:

```
cost = norm(qTest - q0);
```

This heuristic significantly improves motion quality in practice. By preferring the configuration closest to the robot's current posture, the controller reduces unnecessary joint excursions and avoids many of the strange or unsafe configurations that were observed in earlier tests.

7.5. Approach Strategy for Safer Picking

Another important correction introduced in the final implementation concerns how the robot approaches the target position.

Instead of moving directly to the final object position, the controller first computes an intermediate point located slightly behind the target along the radial direction in the XY plane:

```
dir = p_robo(1:2) / norm(p_robo(1:2));
dir3 = [dir 0];

d = 0.10;
p_novo = p_robo - d * dir3;
p_novo(3) = p_robo(3);
```

This intermediate point allows the robot to first align itself with the angular direction of the target and then move approximately straight toward the final object position.

In practice, this strategy was adopted to avoid approaching the object in an unsafe manner. Without this staged approach, the robot could reach the target with a trajectory that risks pushing, hitting, or crushing the object before the grasp is performed. By first positioning itself at a nearby approach point and only then advancing toward the final target, the pick motion becomes more controlled and reliable.

7.6. Command Execution

The control loop therefore executes motion in two stages:

1. Move to the intermediate approach position.
2. Move from the approach position to the final target position.

The corresponding joint configurations are computed using `computeIK_UR5`, and the commands are sent through:

```
sendJointConfigurationAndWait(ctx.ur, qSol, 'EndTime', 10);
```

for the approach motion, and then:

```
sendJointConfigurationAndWait(ctx.ur, qSol, 'EndTime', 5);
```

for the final motion.

Delays are inserted between steps to ensure proper synchronization and to allow the robot to stabilize before continuing:

```
pause(3);
```

If the inverse kinematics solver does not return a valid solution, the command is aborted and no motion is sent to the robot.

7.7. Tool Output and Gripper Actuation

The MATLAB node also creates a ROS2 service client to control the robot tool output:

```
ioClient = ros2svcclient(rosNode, ...
    "/io_and_status_controller/set_io", ...
    "ur_msgs/SetIO");
```

The following part of the code is intended to actuate the gripper:

```
% setToolOutput(ioClient, 1.0);
% pause(2);
% setToolOutput(ioClient, 0.0);
```

This sequence is used to close the robot gripper. However, during the experiments, the physical cable connected between the robot I/O and the gripper presented intermittent problems. For this reason, this part of the code was left commented during testing.

To enable the gripping action, the physical connection must first be verified. In particular, it is necessary to ensure that the cable is correctly connected and that the gripper is properly powered and responsive (for example, by checking whether its LED indicator is on). Once the hardware connection is confirmed to be working correctly, this section of the code can be uncommented to enable the grasping command.

7.8. Integration with Camera-Based System

In addition to the cage-based instruction system, the control pipeline was also tested with a camera-based perception module.

Unlike the calibration procedure described in Section 6, no additional transformation or AX = XB calibration was required in this case. The camera group provided object coordinates already expressed in the robot base reference frame.

As a result, the MATLAB controller did not perform any coordinate conversion. The received position was directly used as input for inverse kinematics and motion execution.

From a ROS2 perspective, the integration remained essentially the same: the MATLAB node subscribed to a topic publishing 3D positions, extracted the translational components, and used them as target positions for the robot.

7.9. Overall Control Logic

The final control loop can be summarized as follows:

1. Receive target positions through ROS2.
2. Accept only points that satisfy the throttling condition.
3. Store the latest valid point.
4. Wait for manual user confirmation.
5. Compute an intermediate approach point.
6. Solve inverse kinematics using multiple candidate seeds.
7. Select the most suitable joint solution.
8. Execute the approach motion.
9. Execute the final motion toward the target.
10. Optionally actuate the gripper through the tool output.

This final implementation reflects the practical corrections introduced during development. Compared with earlier versions, it provides more stable IK behavior, safer approach trajectories, and better consistency during real robot execution.

8. ROS2 Integration

In this project, ROS2 is used to enable communication between two separate machines: a Python-based system responsible for perception and high-level instruction, and a MATLAB-based system responsible for robot control.

The MATLAB machine acts primarily as a ROS2 subscriber and service client, receiving target positions and issuing control commands to the UR5 robot.

8.1. Network Configuration

To establish communication between the two machines, both systems must:

- Be connected to the same local network,
- Belong to the same IP subnet,
- Share the same ROS2 Domain ID.

The IP address of each machine was verified using:

```
ip a
```

To ensure that both machines operate within the same DDS domain, the following environment variable was configured:

```
export ROS_DOMAIN_ID=0
```

In MATLAB, the same configuration is enforced using:

```
setenv('ROS_DOMAIN_ID','0');
setenv('ROS_LOCALHOST_ONLY','0');
```

Once correctly configured, ROS2 automatically handles node discovery via DDS, without requiring a centralized master node.

8.2. ROS2 Initialization in MATLAB

Before creating nodes or subscribers, MATLAB must be configured to recognize the ROS2 environment and custom message definitions:

```
folder = fullfile(getenv("HOME"), "pie_ws", "src");
ros2genmsg(folder);
savepath;
rehash toolboxcache;
```

This step ensures that any custom messages in the ROS2 workspace are properly available within MATLAB.

A dedicated ROS2 node is then created:

```
rosNode = ros2node("/matlab_ur5_node");
```

8.3. Subscriber Implementation

The MATLAB node subscribes to the topic published by the external system:

```
sub = ros2subscriber(rosNode, "/instruction_cage", ...
    "geometry_msgs/Transform", ...
    @(msg) storePointThrottled(ctx, msg));
```

The message type `geometry_msgs/Transform` is used to transmit the target position. In the final system, only the translational component is used:

```
x = msg.translation.x;
y = msg.translation.y;
z = msg.translation.z;
```

The received position is already expressed in the robot base frame and is stored for later execution.

8.4. Callback-Based Data Handling

Instead of synchronous message retrieval, the system uses a callback-based approach. The subscriber invokes a function every time a new message is received:

```
@(msg) storePointThrottled(ctx, msg)
```

This function performs the following operations:

- Applies a throttling condition to limit update frequency,
- Stores the most recent valid position,
- Updates shared state variables used by the main control loop.

This design enables asynchronous data reception while maintaining control over when commands are actually executed.

8.5. Service Client for Tool Control

In addition to subscribing to position data, the MATLAB node also creates a ROS2 service client to control the robot tool output:

```
ioClient = ros2svcclient(rosNode, ...
    "/io_and_status_controller/set_io", ...
    "ur_msgs/SetIO");
```

This service allows controlling digital outputs of the robot, which can be used to actuate external devices such as a gripper.

Before use, the client waits for the service to become available:

```
waitForServer(ioClient);
```

Although the actuation commands were implemented, they were not used in the final experiments due to hardware issues with the physical connection to the gripper.

8.6. Communication Verification

To validate correct communication between systems, the following commands were used:

```
ros2 topic list
ros2 topic echo /instruction_cage
```

Successful visualization of incoming messages confirmed:

- Proper DDS discovery,
- Correct topic configuration,
- Successful data transmission between machines.

8.7. Role in the Control Architecture

The ROS2 integration plays a central role in the system:

- It enables real-time transmission of target positions from the perception module to the control module,
- It decouples perception (Python) from control (MATLAB), improving modularity,
- It allows asynchronous data acquisition combined with controlled execution,
- It provides an interface for extending the system with additional components (e.g., gripper control).

This architecture ensures a flexible and scalable communication framework, allowing independent development and testing of perception and control subsystems.

9. Conclusion

This project presented the development of a complete control and communication pipeline for the UR5 robotic manipulator, integrating MATLAB-based control with ROS2-based communication and a vision-driven input system.

The final implementation successfully enabled the robot to receive target positions from an external system and execute corresponding motions using inverse kinematics. The integration between MATLAB and ROS2 proved to be effective, allowing reliable data exchange between separate machines and supporting a modular system architecture.

Throughout the development process, several practical challenges were identified and addressed. In particular, improvements to the inverse kinematics solution selection significantly enhanced motion stability, avoiding undesirable configurations such as self-collisions and abrupt joint movements. Additionally, the introduction of an intermediate approach strategy allowed safer and more controlled interaction with target objects.

The calibration procedure based on the AX = XB formulation provided a consistent transformation between the camera and robot reference frames, enabling proper alignment between perception and control. Although this transformation was later incorporated upstream in the system, its validation was essential during the testing phase.

Some limitations were encountered during the experiments. In particular, hardware issues related to the gripper connection prevented full validation of the grasping stage. Furthermore, the system currently relies on manual triggering of robot motions, which limits full autonomy.

Future work may include improving the robustness of the perception pipeline, enabling fully autonomous execution, refining calibration accuracy, and restoring reliable gripper actuation. These improvements would allow the system to evolve into a fully operational pick-and-place solution.

Overall, the project demonstrates a functional and extensible framework for integrating robotic control, perception, and communication, providing a solid foundation for further development and experimentation.